

Chapter 3

Memory Management

FIFO Page Replacement Algorithm

Always replace the oldest page.

“Replace the page that has been in memory for the longest time.”

Maintain a linked list of all pages in memory

Keep in order of when they came into memory

The page at the front of the list is oldest

Add new page to end of list

Disadvantage:

The oldest page may be needed again soon

Some page may be important

**It will get old, but replacing it
will cause an immediate Page Fault**

Second Chance Page Replacement Algorithm

Modification to FIFO

Pages kept in a linked list

Oldest is at the front of the list

Look at the oldest page

If its “referenced bit” is 0...

Select it for replacement

Else

It was used recently; don’t want to replace it

Clear its “referenced bit”

Move it to the end of the list

Repeat

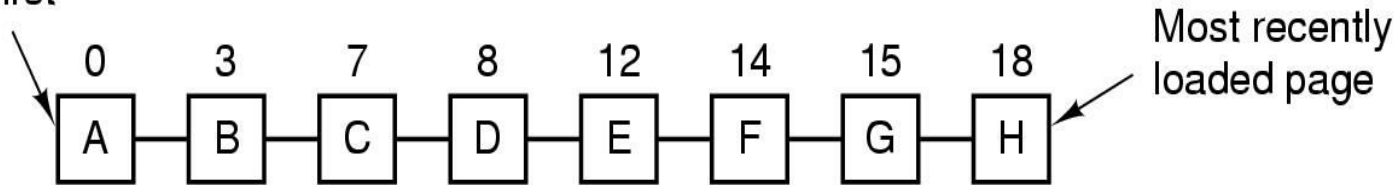
Everything was used in last clock tick?

Eventually we will get back to the oldest page

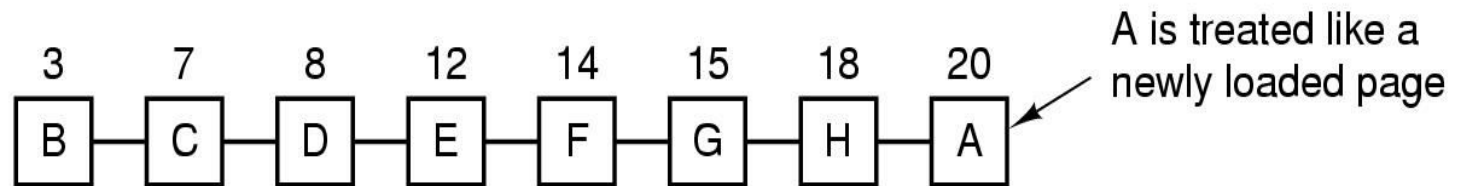
This time, its ref. bit will be 0 and we’ll select it.

Second Chance Page Replacement Algorithm

Page loaded first



(a)



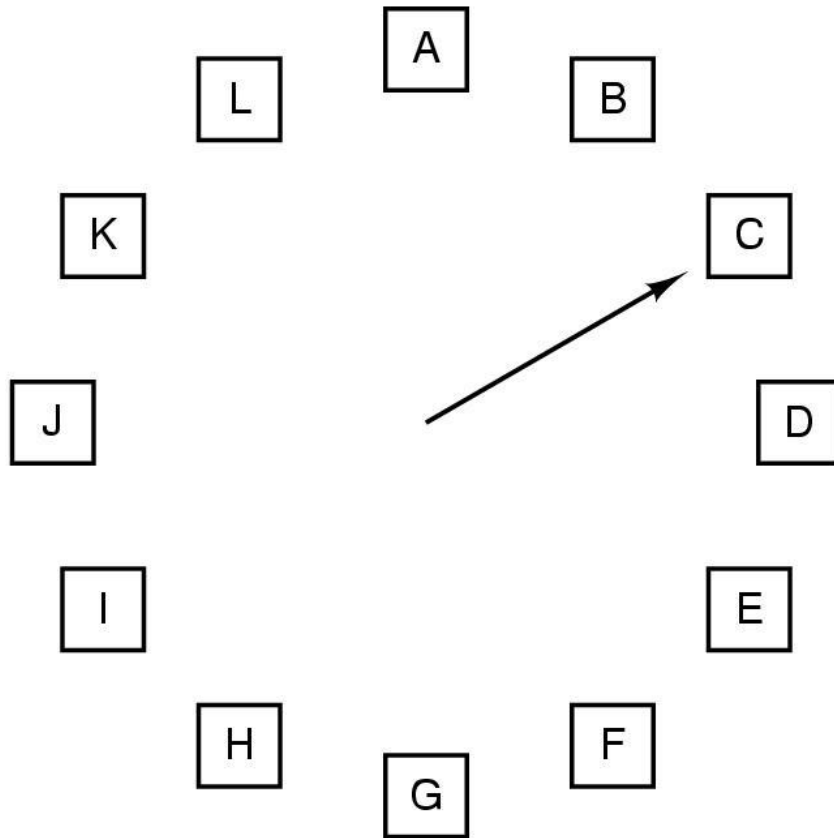
(b)

- Operation of a second chance
 - Page list if **fault** occurs at time 20, A has *R* bit set (numbers above pages are loading times)

The Clock Page Replacement Algorithm

- Efficient version of Second-chance
 - pages don't have to be constantly pushed to the back of the list
 - keeps a circular list of pages in memory
 - the hand (iterator) points to
 - The oldest page (initially)
 - the last examined page

The Clock Page Replacement Algorithm



When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

- R = 0: Evict the page
- R = 1: Clear R and advance hand
and the process is repeated until a page is replaced

Least Recently Used (LRU)

- Assume pages used **recently** will be used **again** soon
 - throw out page that has been **unused** for **longest** time

Implementation #1:

Keep a linked list of all pages

On every memory reference,

Move that page to the front of the list.

The page at the tail of the list is replaced.

“on every memory reference...”

Not feasible in software

Least Recently Used (LRU)

- Assume pages used **recently** will be used **again** soon
 - throw out page that has been **unused** for **longest** time

Implementation #2:

MMU maintains a **counter**

Incremented on every clock cycle

Every time a page table entry is used

MMU writes the value to the entry

“timestamp” / “time-of-last-use”

When a page fault occurs

Software looks through the page table

Identifies the entry with the oldest timestamp

Implementation #3:

	Page			
	0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

(a)

	Page			
	0	1	2	3
0	0	0	1	1
1	1	0	1	1
2	0	0	0	0
3	0	0	0	0

(b)

	Page			
	0	1	2	3
0	0	0	0	1
1	1	0	0	1
2	1	1	0	1
3	0	0	0	0

(c)

	Page			
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0

(d)

	Page			
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	1
3	1	1	0	0

(e)

0	0	0	0
1	0	1	1
1	0	0	1
1	0	0	0

(f)

0	1	1	1
0	0	1	1
0	0	0	1
0	0	0	0

(g)

0	1	1	0
0	0	1	0
0	0	0	0
1	1	1	0

(h)

0	1	0	0
0	0	0	0
1	1	0	1
1	1	0	0

(i)

0	1	0	0
0	0	0	0
1	1	0	0
1	1	1	0

(j)

LRU using a matrix – pages referenced in order
0,1,2,3,2,1,0,3,2,3

Simulating LRU in Software

1. Not Frequently Used (NFU) Algorithm
2. Aging

Not Frequently Used (NFU) Algorithm

- Associate a counter with each page
- On every timer interrupt, the OS looks at each page.
If the Reference Bit is set...
Increment that page's counter & clear the bit.
- The counter approximates how often the page is used.
- For replacement, choose the page with lowest counter.

Problem:

Some page may be heavily used

---> Its counter is large

The program's behavior changes

Now, this page is not used ever again (or only rarely)

This algorithm never forgets!

This page will never be chosen for replacement!

Aging

- Modification of NFU to make it aware of the time span of use
- Associate a counter with each page
- On every timer interrupt, the OS looks at each page.
 - Shift the counter right 1 bit
(divide its value by 2)
 - If the Reference Bit is set...
 - Set the most-significant bit
 - Clear the Referenced Bit.

Aging

- For instance, if a page has referenced bits 1,0,0,1,1,0 in the past 6 clock ticks, its counter will look like this:
 - 10000000,
 - 01000000,
 - 00100000,
 - 10010000,
 - 11001000,
 - 01100100.
- Page references **closer** to the **present** time have **more impact** than page references long ago

Aging

- Difference with LRU
 - The exact reference time not known
 - Can recall limited past